

Integration of Rule-Based Filtering and Stacking Ensemble for Phishing Website Detection

Sarah Rosdiana Tambunan ¹⁾, Nikita Simanjuntak ²⁾,
Grace Arintya Siahaan ³⁾, Ruben Martogi Jeremia Sianipar ⁴⁾

Information System, Faculty of Informatics and Electrical Engineering,
Del Institute of Technology, Jl Sisingamangaraja Laguboti,
Toba, North Sumatera 22381, Indonesia
Email : sarah.tambunan@del.ac.id

Riwayat Artikel		
Diterima:	Direvisi:	Disetujui:
01-01-2024	01-02-2024	01-03-2024

Abstract

Phishing attacks targeting financial and banking sectors continue to increase, requiring detection systems that are not only accurate but also efficient and interpretable. Existing approaches rely predominantly on single models with simple ensemble mechanisms and lack an early filtering layer, resulting in limited generalization and high computational burden. A hybrid architecture is proposed, integrating Rule-Based Filtering as an initial screening layer, a Stacking Ensemble combining Random Forest and XGBoost with Logistic Regression as meta-learner, and an explainability module using SHAP and Large Language Model to generate natural language justifications for each prediction. Hyperparameter optimization was performed using a Genetic Algorithm. Controlled evaluation on the test dataset demonstrates that the Stacking Ensemble achieves the best balanced performance in terms of accuracy and discriminative ability, while the hybrid model reduces machine learning computational load with improved latency consistency. Real-world testing on active URLs further confirms system effectiveness across both controlled and operational conditions. The alignment between SHAP-identified dominant features and Rule-Based Filtering indicators validates the methodological coherence of the proposed architecture. These findings demonstrate that the integration of deterministic filtering, adaptive ensemble learning, and explainability layers produces a phishing detection system that is accurate, computationally efficient, and capable of providing transparent and human-readable decision justifications.

Keywords: *phishing detection, stacking ensemble, rule-based filtering, explainable artificial intelligence, large language model.*

Abstrak

Serangan *phishing* yang terus meningkat pada sektor keuangan dan perbankan memerlukan sistem deteksi yang tidak hanya akurat, tetapi juga efisien dan dapat diinterpretasikan. Pendekatan yang ada saat ini masih mengandalkan model tunggal dengan mekanisme penggabungan sederhana dan tidak memiliki lapisan penyaringan awal, sehingga kemampuan generalisasi terbatas dan beban komputasi tinggi. Arsitektur hibrid diusulkan dengan mengintegrasikan Rule-Based Filtering sebagai lapisan penyaringan awal, Stacking Ensemble yang menggabungkan Random Forest dan XGBoost dengan Logistic Regression sebagai meta-learner, serta modul interpretabilitas berbasis SHAP dan Large Language Model untuk menghasilkan justifikasi prediksi dalam bahasa alami. Optimasi hyperparameter dilakukan menggunakan Genetic Algorithm. Evaluasi terkontrol pada data uji menunjukkan bahwa Stacking Ensemble menghasilkan keseimbangan performa terbaik dari sisi akurasi dan kemampuan diskriminatif, sementara model hibrid berhasil mereduksi beban komputasi machine learning dengan konsistensi latensi yang lebih baik. Pengujian operasional pada URL aktif mengonfirmasi efektivitas sistem pada kondisi nyata. Keselarasan antara fitur dominan hasil analisis SHAP dan indikator Rule-Based Filtering memvalidasi koherensi metodologis arsitektur yang diusulkan. Temuan ini menunjukkan bahwa integrasi penyaringan deterministik, ensemble learning adaptif, dan lapisan interpretabilitas menghasilkan sistem deteksi *phishing* yang akurat, efisien secara komputasi, serta mampu menyediakan justifikasi keputusan yang transparan dan dapat dipahami pengguna.

Kata kunci: deteksi phishing, stacking ensemble, pemfilteran berbasis aturan, kecerdasan buatan yang dapat dijelaskan, large language model.

Introduction

Phishing attacks represent one of the most significant cybersecurity threats, constituting a form of cybercrime that exploits social engineering techniques to deceive victims into accessing counterfeit websites and surrendering sensitive information such as account credentials and financial data [1]. According to the Anti-Phishing Working Group (APWG), 1,003,924 phishing attacks were recorded globally in the first quarter of 2025 the highest figure since late 2023 with a gradual upward trend from 877,536 incidents in early 2024 to 989,123 incidents by year-end [2]. Financial services and banking sectors remain the primary targets, accounting for 30.9 percent of all attacks [3]. This urgency drives the development of detection systems that are accurate, adaptive, and computationally efficient [4].

Machine learning approaches have been widely applied to phishing detection and have demonstrated strong performance across various studies [5]. Prior work has compared multiple classification algorithms—including Gradient Boosting, Random Forest, Decision Tree, SVC, and K-Nearest Neighbors—with the first two consistently exhibiting superior performance [6]. Other studies have established that Random Forest

and XGBoost outperform Support Vector Machine in phishing detection accuracy [7]. However, the majority of these studies share several fundamental limitations: single models are susceptible to high bias-variance trade-offs that constrain generalization [6] [8], model combination mechanisms remain simplistic, relying on probability averaging without leveraging adaptive meta-learning [9], and no early rule-based filtering layer exists to reduce the computational burden of machine learning and minimize detection errors at the initial stage of the pipeline [10].

Although improving classification performance has been the primary focus of much prior work, model interpretability has equally emerged as a critical concern in phishing detection system development [11]. Ensemble models such as Random Forest and XGBoost typically produce predictions whose decision-making processes cannot be directly observed [12]. Explainable Artificial Intelligence (XAI) methods such as SHapley Additive exPlanations (SHAP) have been widely adopted to explain feature contributions to prediction outcomes [13]; however, the resulting interpretations remain predominantly in the form of numerical representations and technical visualizations that are not readily comprehensible to non-technical users [14][15]. Against this backdrop, a research gap persists: no existing architecture integrates rule-based filtering, ensemble learning that simultaneously leverages the strengths of multiple algorithms, and an interpretability mechanism capable of translating prediction outputs into natural language explanations within a unified system framework.

The proposed hybrid phishing detection architecture integrates three complementary components: first, Rule-Based Filtering as an initial layer employing deterministic rules grounded in validated phishing indicators; second, a Stacking Ensemble combining Random Forest as a bagging algorithm and XGBoost as a boosting algorithm, with Logistic Regression as the meta-learner; and third, an Explainable AI module utilizing SHAP to explain individual feature contributions to each prediction, augmented by a Large Language Model (LLM) to generate automatic textual reasoning. Hyperparameter optimization is performed using a Genetic Algorithm to explore a broader search space than conventional grid search. The primary contributions include the design of a hybrid phishing detection architecture that integrates Rule-Based Filtering as an early screening layer, Stacking Ensemble as the classification core, and a SHAP–LLM module as an interpretability layer that translates model decisions into natural language explanations. This study also presents a quantitative analysis of performance trade-offs among accuracy, recall, computational efficiency, and interpretability based on evaluation across a benchmark dataset and operational testing scenarios.

Research Method

A machine learning-based experimental approach was employed to detect phishing URLs through the integration of Rule-Based Filtering and a Stacking Ensemble, with the architecture illustrated in Figure 1. The system workflow begins when a URL is received as input and its features are extracted, after which it proceeds to an initial screening stage using Rule-Based Filtering; URLs that cannot be directly classified are forwarded to the Stacking Ensemble model, consisting of Random Forest and XGBoost as base learners with Logistic Regression as the meta-learner. The prediction results are subsequently analyzed using SHAP to generate feature contribution values, which are then translated by a Large Language Model (LLM) into natural-language explanations. The contribution of each component was evaluated through four experimental scenarios: standalone Random Forest, standalone XGBoost, a Stacking Ensemble without Rule-Based Filtering, and the hybrid system combining both.

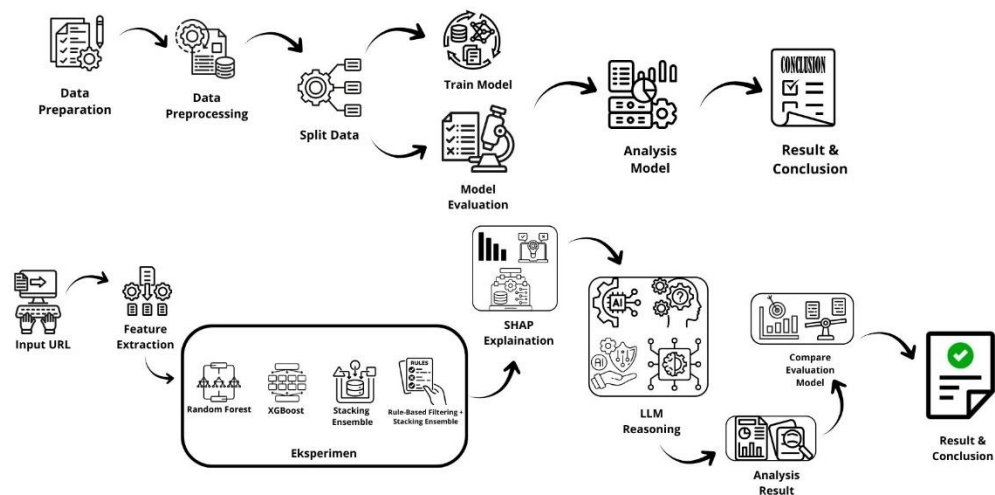


Figure 1 Research Architecture

The dataset used is the dataset_phishing sourced from the public Kaggle repository, consisting of 11,430 samples with 89 features and a balanced class distribution between phishing and legitimate classes. Six features were eliminated due to zero variance and irrelevance to the URL detection scenario, and one duplicate record was removed, leaving 83 active features from 11,429 clean samples, which were split using an 80:20 ratio for training and testing. Feature extraction is adaptive: when a web page can be successfully accessed, 81 hybrid features are extracted, encompassing URL structural features, page content, and domain information; when

the page is inaccessible, the system switches to a fallback mode using 37 URL-structure-based features only. Both feature sets are subsets of the 83 features in the main dataset, ensuring direct compatibility with the trained models.

The first layer of the pipeline is Rule-Based Filtering, which functions as a deterministic screening mechanism to identify suspicious URLs prior to machine learning analysis. Of the 83 available features, 24 were selected as the basis for rule formulation based on their ability to explicitly represent phishing characteristics, their consistency of occurrence in phishing URLs, and their ease of formulation into deterministic rules; features that are statistical in nature or require learning non-linear relationships remain utilized at the machine learning stage. All 24 rules, along with their categories and descriptions, are presented in Table 1.

Table 1 Aturan Rule-Based Filtering

No	Rule	Category	Description
1	suspicious_tld	Critical	TLD flagged as suspicious. Includes country-code TLDs, new generic TLDs (New gTLD), and TLDs with poor reputation.
2	random_domain	Critical	Random or meaningless domain names indicating automatic domain generation for phishing activity.
3	ip	Critical	Hostname using an IP address instead of a domain name is considered a phishing indicator.
4	shortening_service	Critical	URL using a shortening service (bit.ly, tinyurl, etc.) is flagged as suspicious.
5	http_in_path	Critical	Occurrence of 'http' in the URL path indicates URL obfuscation.
6	nb_subdomains	Important	> 3 subdomains flagged as suspicious.
7	nb_dots	Important	> 4 dot characters (.) flagged as suspicious.
8	nb_slash	Important	> 7 slash characters (/) flagged as suspicious.
9	length_hostname	Important	> 30 hostname character length flagged as suspicious.
10	nb_at	Important	>= 1 '@' character flagged as suspicious; '@' before hostname flagged as phishing.
11	nb_percent	Important	> 5 percent characters (%) flagged as suspicious.
12	nb_tilde	Important	>= 1 tilde character (~) flagged as suspicious.
13	nb_semicolon	Important	>= 1 semicolon character (;) flagged as suspicious.
14	nb_star	Important	>= 1 star character (*) flagged as suspicious.
15	nb_comma	Important	>= 1 comma character (,) flagged as suspicious.
16	nb_dollar	Important	>= 1 dollar sign character (\$) flagged as suspicious.
17	nb_qm	Important	> 2 question mark characters (?) flagged as suspicious.
18	nb_colon	Important	> 1 colon character (:) (beyond the protocol) flagged as suspicious.
19	nb_eq	Important	> 8 equal sign characters (=) flagged as suspicious.
20	nb_and	Important	> 3 ampersand characters (&) flagged as suspicious.
21	nb_hyphens	Important	> 3 hyphen characters (-) flagged as suspicious.
22	nb_underscore	Important	> 3 underscore characters (_) flagged as suspicious; underscore in domain name flagged as highly suspicious.

23	ratio_digits_url	Moderately Important	Proportion of digits in hostname. > 0.3 (more than 30% of URL consists of digits) flagged as suspicious.
24	port	Moderately Important	Use of non-default ports (Standard Ports: 21, 22, 23, 80, 443, 445, 1433, 1521, 3306, 3389) flagged as suspicious.

The rules in Table 1 are grouped into three categories based on their level of contribution to phishing risk: the Very Important category comprises four features with a strong association with phishing indicators rarely found in legitimate URLs; the Important category comprises 17 features that are common on phishing sites but require a combination of features to increase classification confidence; and the Fairly Important category comprises two supporting features that complement the overall risk assessment. The decision-making mechanism operates in two stages: a High-Confidence Rule, which directly classifies a URL as phishing if at least one Very Important feature is detected, and a Threshold-Based Rule, applied when no such indicator is found, with a risk score calculated using a weight of two for Important-category features and a weight of one for Fairly Important-category features.

Based on this risk score, URLs are classified into three categories as shown in Table 2. URLs with a score greater than or equal to five are directly classified as phishing, while scores of one to four are categorized as suspicious and a score of zero as benign; the latter two categories are forwarded to the machine learning stage, as their indicators are not yet conclusive enough to produce a deterministic decision.

Table 2 Rule-Based Filtering Score Categories

<i>Risk Score</i>	<i>Categories</i>
≥ 5	<i>Phishing</i>
1-4	<i>Suspicious</i>
0	<i>Benign</i>

At the machine learning stage, Random Forest and XGBoost were employed as base learners due to their complementary characteristics: Random Forest reduces variance through a bagging mechanism, while XGBoost adaptively reduces bias through gradient boosting reinforced by L1 (reg_alpha) and L2 (reg_lambda) regularization to control overfitting. Hyperparameter optimization for both models was performed using a Genetic Algorithm (GA) with F1-Score as the fitness function, with the search space encompassing n_estimators, max_depth, and max_features for Random Forest, and learning_rate, max_depth, subsample, and colsample_bytree for XGBoost; the optimal configuration resulting from GA was subsequently employed in the Stacking Ensemble architecture, with Logistic Regression as the meta-learner receiving the predict_proba outputs of both base learners.

SHAP was applied at two levels: globally, using summary plots and bar plots of mean absolute SHAP values, and locally, using force plots for individual URL predictions; TreeExplainer was used for Random Forest and XGBoost, while KernelExplainer was used for Logistic Regression. The SHAP contribution values were subsequently passed to the LLM together with the predicted label and probability value to generate natural-language explanatory narratives. Evaluation was conducted in two stages: a controlled evaluation on the test dataset using accuracy, precision, recall, F1-Score, and AUC metrics accompanied by confusion matrices, and an operational evaluation using 40 active URLs (20 benign and 20 phishing) to measure the performance gap between controlled and real-world conditions. All four experimental scenarios were evaluated at both stages to enable a systematic comparison of the contribution of each architectural component.

Result and Discussion

- **Model Performance Evaluation**

Testing was conducted on eight model variants grouped according to two feature extraction modes: a 37-feature mode based on URL structure, used as a fallback when a page cannot be accessed, and an 81-feature hybrid mode combining URL, page content, and domain information features. Hyperparameter optimization using the Genetic Algorithm produced significantly different configurations between Random Forest and XGBoost, as shown in Tables 3 and 4.

Table 3 Random Forest Hyperparameters

Hyperparameter	Nilai
n_estimators	464
max_depth	14
min_samples_split	4
min_samples_leaf	1
max_features	log2
random_state	12

Table 4 XGBoost Hyperparameters

Hyperparameter	Nilai
n_estimators	372
learning_rate	0,1266
max_depth	5
subsample	0,7610
colsample_bytree	0,7113
min_child_weight	3,8671

gamma	0,4511
reg_alpha	0,6643
reg_lambda	2,2702
random_state	12

A comparison of the performance of all models based on accuracy, precision, recall, F1-score, and AUC metrics is presented in Figures 2 and 3.

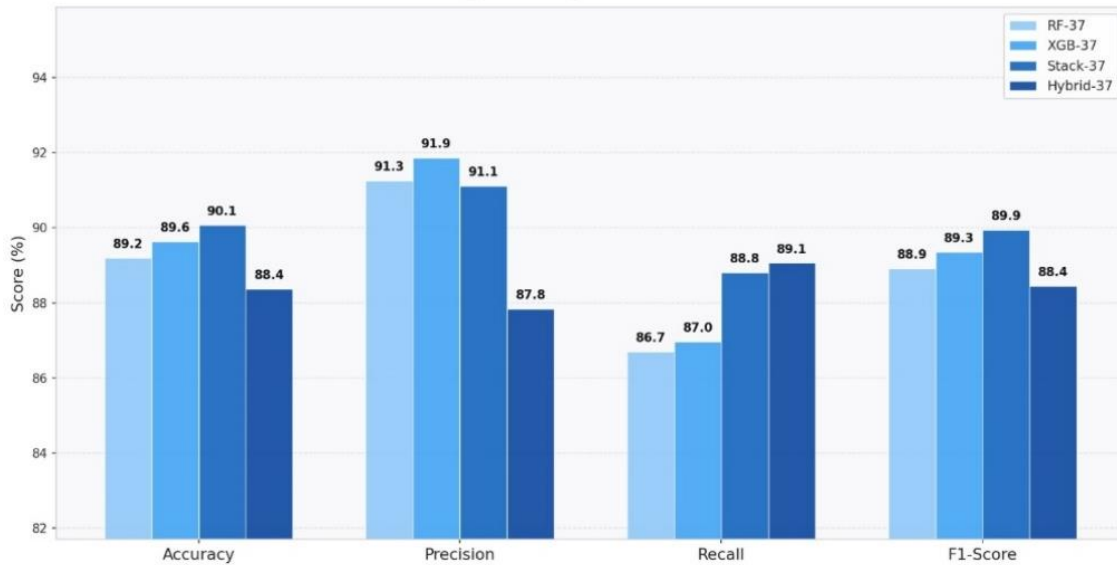


Figure 2 Metric Evaluation 37 Feature Model

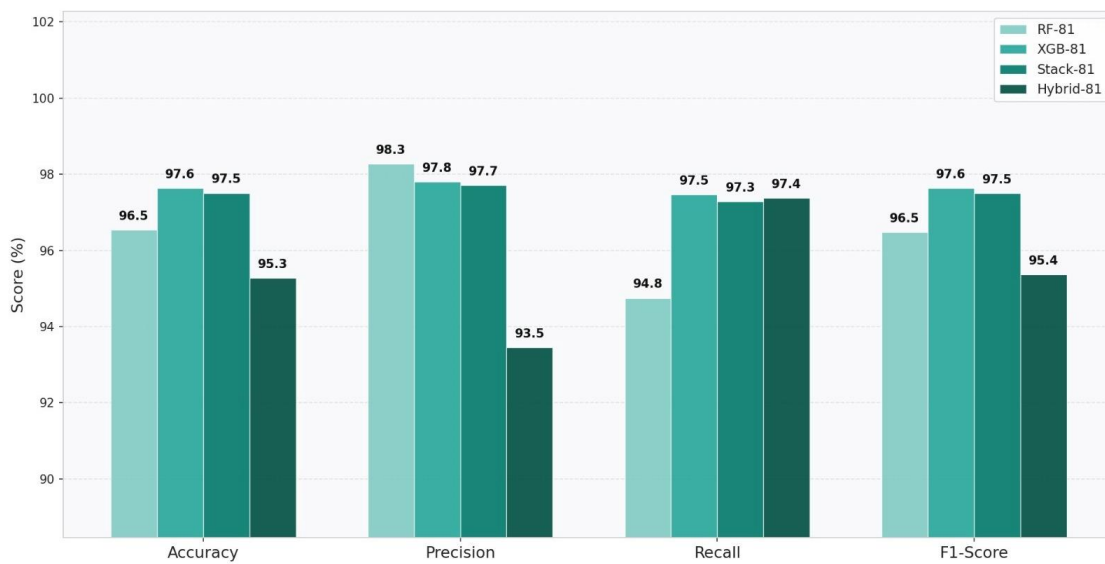


Figure 3 Metric Evaluation 81 Feature Model

The most striking difference is the consistent six- to seven-percentage-point gap between the 81-feature and 37-feature model groups across all evaluation metrics, confirming that web page content features provide a significant discriminative contribution in distinguishing phishing sites from legitimate sites that are structurally similar in URL form. Within the 81-feature group, Random Forest achieved the highest precision (98.3 percent) but lower recall (94.8 percent), reflecting the conservative characteristics of bagging in labeling URLs as phishing. XGBoost recorded the best overall performance, with an accuracy of 97.64 percent, F1-Score of 97.63 percent, and AUC of 99.69 percent, accompanied by a more balanced error distribution (FP = 25, FN = 29); this advantage indicates that the boosting mechanism, which iteratively corrects residuals, more effectively handles the complexity of phishing patterns than the independent aggregation performed by bagging.

The Stacking Ensemble achieved performance comparable to XGBoost (accuracy 97.51 percent, AUC 99.63 percent) with a better balance of FP and FN (FP = 26, FN = 30). Although it did not surpass XGBoost as a single model a phenomenon commonly observed when one base learner already achieves very high performance the Stacking Ensemble remains more suitable for systems that prioritize minimizing false negatives in a threat detection context. A comparison of classification error distributions across all models is presented in Tables 5 and 6. The FP = 25 and FN = 29 figures for XGBoost in the 81-feature mode reflect the most balanced error distribution among all models, consistent with the characteristics of gradient boosting, which produces more precise decision boundaries than bagging.

Table 5 Combined Confusion Matrix for 81 Feature Model

Model	TN	FP	FN	TP
Random Forest	1.124	19	60	1.083
XGBoost	1.118	25	29	1.114
Stacking	1.117	26	30	1.112
Hybrid	1.065	78	30	1.113

Table 6 Confusion Matrix for 37 Feature Model

Model	TN	FP	FN	TP
Random Forest	1.048	95	152	991
XGBoost	1.055	88	149	994
Stacking	1.044	99	128	1.015
Hybrid	1.002	141	125	1.018

The application of Rule-Based Filtering within the Hybrid architecture produces a dual impact that warrants careful analysis. Of the total 2,286 test URLs, 560

URLs (24.5 percent) were directly filtered without involving machine learning computation. The Hybrid model achieved the highest recall, at 97.81 percent, but with a corresponding decrease in precision to 88.52 percent due to an increase in FP to 145 nearly five times higher than Stacking. This phenomenon stems from the deterministic nature of heuristic rules, which cannot distinguish between benign URLs with suspicious technical characteristics and genuine phishing URLs, and decisions made at the rule-based layer cannot be corrected by the Stacking Ensemble. Within the defense-in-depth principle of cybersecurity, this trade-off is acceptable, as failing to detect phishing (false negatives) is generally far more damaging than excessive warnings (false positives).

• Computational Efficiency and Consistency Results

From a computational efficiency perspective, Rule-Based Filtering reduced the machine learning workload by 24.5 percent, increased throughput from 7,542 to 7,750 URLs per second, and reduced batch processing time from 0.303 to 0.295 seconds, as shown in Table 7. Although the 2.7 percent improvement appears moderate at this experimental scale, its impact would become increasingly significant in large-scale production systems, as early filtering reduces the cumulative load proportionally to data volume. More operationally relevant, the Hybrid model exhibited a lower latency standard deviation (4.577 ms) compared to Stacking (6.985 ms), indicating better prediction consistency. In real-time cybersecurity detection systems, response-time consistency is often more valuable than average speed alone, as unpredictable latency spikes can disrupt system stability.

Table 7 Model Processing Efficiency Comparasion

Aspect	Stacking	Hybrid	Analysis
Total URLs	2,286	2,286	Identical
URLs processed by Rule-Based	0 (0%)	560 (24.5%)	Hybrid reduces ML workload
URLs processed by ML	2,286 (100%)	1,726 (75.5%)	Computational load reduction
Batch Time (seconds)	0.1972	0.1736	12.0% faster
Throughput (URL/second)	11,593.1	13,164.7	Hybrid slightly higher

Table 8 Predetection Latency per URL

Model	Latency (ms)	Min (ms)	Max (ms)	Std (ms)
Random Forest	117.247	72.672	159.430	23.000
XGBoost	12.426	9.623	22.808	3.166
<i>Stacking ensemble</i>	134.313	113.274	241.678	31.367
Hybrid	119.318	102.043	137.765	10.880

• Model Interpretability Result

SHAP analysis in Figure 4 shows that domain reputation and web page characteristic features contribute most heavily to both models: `google_index` and `page_rank` dominate Random Forest, while `page_rank` and `google_index` also rank highest in XGBoost, followed by `nb_www`, `nb_hyperlinks`, and `phish_hints`. This consistency confirms that non-URL features such as `page_rank`, `google_index`, `web_traffic`, and `nb_hyperlinks` are more discriminative than URL structural characteristics alone, also explaining why the 81-feature model consistently outperforms the 37-feature model across all evaluation metrics.



Figure 4 Top SHAP Contribution Features

SHAP analysis shows that several features used in Rule-Based Filtering, such as `phish_hints`, `nb_hyphens`, `length_hostname`, and `https_token`, are also among the feature groups contributing to the classification process. This finding indicates an alignment between the domain-knowledge-based indicators used at the initial screening stage and the patterns learned by the machine learning model. Thus, Rule-Based Filtering not only functions as an initial screening mechanism but also leverages characteristics proven relevant through model interpretation results.

Table 9 Dominant SHAP Features

Fitur	SHAP RF	SHAP XGB	SHAP STACKING	Status Rule
page_rank	0.0749	1.5294	0.1092	X Not yet
google_index	0.0888	1.1084	0.0945	X Not yet
nb_www	0.0358	0.9198	0.0472	X Not yet
nb_hyperlinks	0.0418	0.7248	0.0453	X Not yet
phish_hints	0.0195	0.5188	0.0242	X Not yet
domain_age	0.0220	0.4267	0.0218	X Not yet
web_traffic	0.0391	0.3631	0.0287	X Not yet

The table above shows seven features that consistently appear in the top ten across all three models simultaneously. These seven features `page_rank`, `google_index`, `nb_www`, `nb_hyperlinks`, `phish_hints`, `domain_age`, and `web_traffic` are entirely absent from the 24 rules currently in the Rule-Based Filtering layer. This finding presents a concrete opportunity to strengthen the Rule-Based Filtering layer: the `phish_hints` feature could be formulated as a rule based on detecting phishing-related keywords in

the URL structure, while domain_age could be formulated as a minimum domain-age threshold rule serving as a suspicion indicator. Integrating these features into the Rule-Based Filtering layer has the potential to increase the coverage of deterministic screening, allowing more phishing URLs to be filtered at the initial stage without relying entirely on machine learning computation.

• LLM Component Analysis Results

The LLM component translates numerical SHAP contribution values into natural-language explanatory narratives. For the legitimate URL(<https://www.ietf.org/>) the model produced a phishing probability of 0.3597, with the contribution toward benign (-0.4717) being more dominant than the contribution toward phishing ($+0.3323$); the LLM translated this result into an explanation stating that the URL was assessed as safe based on high domain reputation and a page structure consistent with legitimate sites. For the phishing URL (<http://nmhokdypni.sum4iit.club/vnafvra97w/?q=3717065149&id=100>), the model produced a probability of 0.7405, dominated by ratio_digits_url ($+0.2324$) and nb_www ($+0.0974$), and the LLM identified the high URL digit ratio and non-standard subdomain usage as the determining factors. The consistency between the SHAP results and the LLM reasoning demonstrates that the system is capable not only of accurate classification but also of providing transparent, understandable justification for each prediction decision for non-technical users. The LLM reasoning output for a sample URL is presented in Table 10.

Table 10 LLM output on sample URLs

AI Reasoning

Hasil deteksi sistem menunjukkan bahwa URL yang diperiksa memiliki probabilitas phishing yang sangat tinggi, yaitu 0.95, yang mana jauh melebihi ambang batas bahaya yang ditentukan (≥ 0.6). Hal ini menunjukkan bahwa URL tersebut sangat mungkin digunakan sebagai sarana penipuan, dan model yang digunakan untuk menganalisisnya telah mendeteksi pola-pola mencurigakan yang umum ditemukan dalam phishing. Dengan demikian, terdapat dukungan kuat untuk kesimpulan akhir bahwa URL ini berbahaya. REKOMENDASI: BLOKIR.

• Experimental Evaluation Results

Operational evaluation was conducted using 40 manually collected active URLs, consisting of 20 legitimate and 20 phishing URLs. Most phishing URLs were inaccessible at testing time, causing the system to operate in the 37-feature fallback mode, a condition that contributed to the performance gap between operational testing and controlled evaluation. A comparison of the performance of the four models is presented in Table 11 and Figure 5.

Table 11 Experiment Confusion Matrix				
Model	TN	FP	FN	TP
Random Forest	17	3	2	18
XGBoost	20	1	5	15
Stacking	18	2	2	18
Hybrid	17	3	3	18

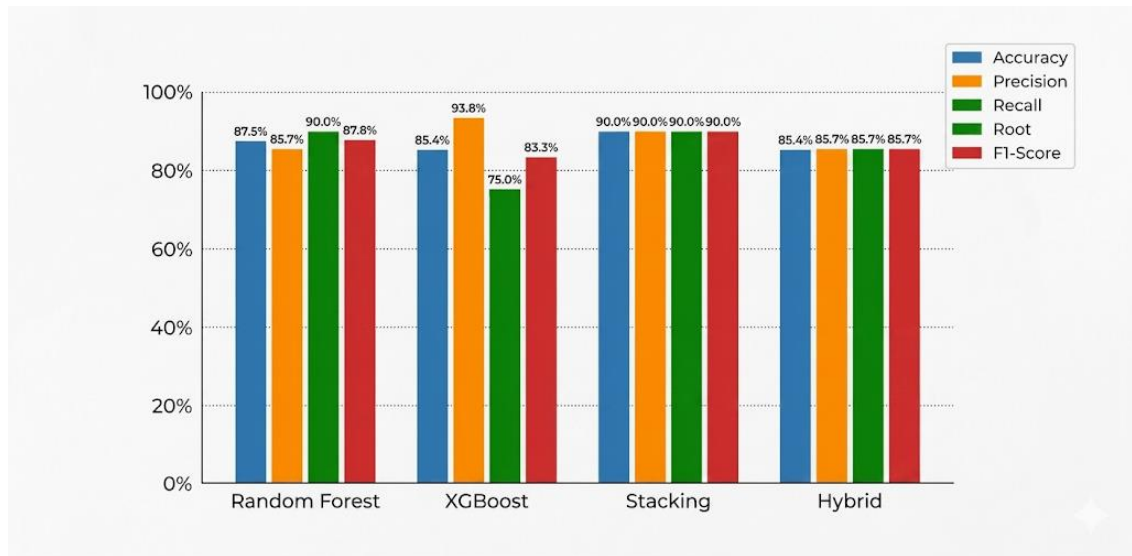


Figure 5 Experiment Metric Evaluation

The Stacking Ensemble achieved the best performance, with an accuracy of 90.0% and the most balanced error distribution ($FP = 2$ and $FN = 2$), indicating that the *meta-learner* successfully compensated for the weaknesses of each *base learner*. Random Forest and Hybrid achieved a high recall (90.0%), but each produced three *false positives*. In contrast, XGBoost generated only one *false positive* but failed to detect five phishing URLs (*false negatives*) due to its more conservative classification behavior. In the context of cybersecurity, failing to detect a phishing URL is considerably more critical than incorrectly classifying a legitimate website as malicious. Therefore, the Stacking Ensemble, which effectively balances both types of errors, is more suitable for real-world operational deployment.

Furthermore, the Rule-Based Filtering component within the Hybrid model successfully identified three phishing URLs directly at the rule layer without requiring any *machine learning* computation, demonstrating the practical contribution of this deterministic component. The performance gap compared with the controlled evaluation across all models is primarily attributed to the reliance on the 37-feature *fallback* mode. This finding suggests that improving feature extraction and incorporating feature enrichment strategies for unresponsive URLs could further enhance the system's performance and robustness.

Overall, the experimental results demonstrate that the proposed hybrid approach, which integrates Rule-Based Filtering with the Stacking Ensemble, effectively reduces computational overhead by filtering URLs that can be deterministically classified before invoking the *machine learning* model, while maintaining high detection performance by minimizing the number of *false negatives*. Consequently, the system not only utilizes computational resources more efficiently but also provides more reliable phishing URL detection, making it better suited for deployment in real-world operational environments.

Conclusion

This study was conducted to address three research questions formulated on the basis of four limitations identified in prior studies in the field of phishing detection: the dominance of single-model architectures vulnerable to bias-variance limitations, model combination mechanisms that remain non-adaptive, the absence of a computationally efficient early filtering layer, and the low interpretability of black-box models, which hinders non-technical users' understanding of prediction decisions. This study proposes a hybrid approach combining Rule-Based Filtering and a Stacking Ensemble optimized through a Genetic Algorithm to improve phishing detection. Beyond performance considerations, this study also integrates SHAP and an LLM to produce more informative and understandable prediction explanations.

The application of Rule-Based Filtering as an initial screening layer proved to have a dual impact on system efficiency and performance consistency. Rule-Based Filtering successfully filtered 24.5 percent of URLs (560 of 2,286) directly without involving machine learning computation, significantly reducing the processing load on the Stacking Ensemble layer. In operational testing on 40 active URLs, 15 phishing URLs were directly identified by the heuristic rule mechanism. The integration of Rule-Based Filtering increased the system's recall to 97.81 percent in the Hybrid model with only 25 false negatives, but at the cost of a significant increase in false positives (FP = 145) due to the deterministic nature of heuristic rules, which cannot be adaptively corrected by the machine learning layer a trade-off that constitutes a critical finding of the proposed architecture.

The integration of SHAP and the LLM proved effective in producing transparent prediction justifications that are understandable to non-technical users. The consistency found between the dominant features identified through SHAP analysis and the indicators used in Rule-Based Filtering further reinforces the overall methodological validity of the architecture. Nevertheless, the role of the LLM in this study remains limited to that of a post-classification explainer based on SHAP output, rather than an active component within the detection process.

Based on the overall findings and identified limitations, three development directions emerge that are directly rooted in the research problem, the methodological choices made, and the resulting analysis. First, the performance gap between test-dataset evaluation and operational conditions, caused by reliance on the 37-feature model when a URL page cannot be accessed, indicates the need to integrate external domain reputation sources such as PhishTank and OpenPhish as additional information layers for unresponsive URLs. Second, the precision trade-off in the Hybrid model arising from the deterministic nature of Rule-Based Filtering indicates the need for a mechanism to dynamically update rules based on model predictions, enabling the system to keep pace with evolving attack patterns. Third, the LLM's role, currently limited to post-classification explanation, opens an avenue for future development toward employing the LLM as an active real-time threat intelligence retrieval agent, with the potential to enhance detection capability against novel phishing attacks not represented in the training data.

Daftar Pustaka

- [1] L. Aziz, F. Ardy, I. Istiqomah, A. E. Ezer, S. N. Neyman, and K. Linux, "Phishing di Era Media Sosial: Identifikasi dan Pencegahan Ancaman di Platform Sosial," no. 4, pp. 1–11, 2024.
- [2] G. Response and T. Cybercrime, "4 Quarter," no. February, 2026.
- [3] E. Mouncey and S. Ciobotaru, "on impact and effectiveness," *J. Econ. Criminol.*, vol. 7, no. December 2024, p. 100125, 2025, doi: 10.1016/j.jeconc.2025.100125.
- [4] F. P. E. Putra, U. Ubaidi, A. Zulfikri, G. Arifin, and R. M. Ilhamsyah, "Analysis of Phishing Attack Trends, Impacts and Prevention Methods: Literature Study," *Brill. Res. Artif. Intell.*, vol. 4, no. 1, pp. 413–421, 2024, doi: 10.47709/brilliance.v4i1.4357.
- [5] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or Not Phishing? A Survey on the Detection of Phishing Websites," *IEEE Access*, vol. 11, no. January, pp. 18499–18519, 2023, doi: 10.1109/ACCESS.2023.3247135.
- [6] A. Khan, D. M. Ahmed, and A. Fathima, "Enhanced Phishing Detection Using Machine Learning Algorithms: A Comparative Study of Random Forest, SVM, and Logistic Regression Models," *SSRN Electron. J.*, 2025, doi: 10.2139/ssrn.5191566.
- [7] V. A. Onih, "Phishing Detection Using Machine Learning: A Model Development and Integration," vol. 07, no. 04, pp. 27–63, 2024.
- [8] A. Khan, M. Ahmed, and A. Fathima, "Enhanced Phishing Detection Using Machine Learning Algorithms: A Comparative Study of Random Forest, SVM, and Logistic Regression Models," 2017.

-
- [9] A. Fajar, S. Yazid, and I. Budi, “Enhancing Phishing Detection through Feature Importance Analysis and Explainable AI : A Comparative Study of CatBoost , XGBoost , and EBM Models”.
 - [10] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, “Machine Learning for Anomaly Detection: A Systematic Review,” *IEEE Access*, vol. 9, pp. 78658–78700, 2021, doi: 10.1109/ACCESS.2021.3083060.
 - [11] Y. Nohara, K. Matsumoto, H. Soejima, and N. Nakashima, “Explanation of Machine Learning Models Using Shapley Additive Explanation and,” vol. 214, no. February, pp. 1–7, 2022.
 - [12] S. S. Shafin, “An explainable feature selection framework for web phishing detection with machine learning,” *Data Sci. Manag.*, vol. 8, no. 2, pp. 127–136, 2025, doi: 10.1016/j.dsm.2024.08.004.
 - [13] M. Perceptron, D. Gaspar, P. Silva, and C. Silva, “Explainable AI for Intrusion Detection Systems : LIME and SHAP Applicability,” *IEEE Access*, vol. PP, no. ML, p. 1, 2024, doi: 10.1109/ACCESS.2024.3368377.
 - [14] M. Li, H. Sun, Y. Huang, and H. Chen, “Shapley value : from cooperative game to explainable artificial intelligence,” vol. 8, 2024, doi: 10.1007/s43684-023-00060-8.
 - [15] L. Hu and K. Wang, “Computing SHAP Efficiently Using Model Structure Information 1,” pp. 1–15.